

Multi-Category Neural Network

Purpose

The purpose of this project is to understand and build a multi-layer neural network and train it to classify handwritten digits into 10 classes (digits 0-9).

Objectives

Learners will be able to

- Understand the role and significance of layers, and various parameters associated with neural network layers.
- Understand the structure and characteristics of a fully connected neural network.
- Understand the importance of activation functions.
- Explore and implement algorithms like Forward and Backward Propagation.
- Implement multi-category classification techniques like Softmax activation and Cross Entropy Loss.

Technology Requirements

- GPU environment (optional)
- Jupyter Notebook
- Python3 (Python 3.8 and above)
- Numpy
- Matplotlib
- [MNIST dataset of handwritten digits](https://huggingface.co/datasets/ylecun/mnist)¹

¹ Y. LeCun, "MNIST Dataset," Hugging Face, [Online]. Available: <https://huggingface.co/datasets/ylecun/mnist>.

Directions

Accessing ZyLabs

You will complete and submit your work through zyBooks's zyLabs. Follow the directions to correctly access the provided workspace:

1. Go to the Canvas project, "**Submission: Multi-Category Neural Network**"
2. Click the "**Load Submission...in new window**" button.
3. Once in ZyLabs, click the **green button** in the Jupyter Notebook to get started.
4. Review the directions and resources provided in the description.
5. When ready, review the provided code and develop your work where instructed.

Project Directions

We will be using the MNIST dataset that contains grayscale samples of handwritten digits of size 28×28 . It is split into a training set of 60,000 examples, and a test set of 10,000 examples.

We split the project into 2 sections.

Section 1

We will define the activation functions and their derivatives which will be used later during forward and backward propagation. We will define the softmax cross entropy loss for calculating the prediction loss.

1. **Activation Functions:** An Activation function usually adds nonlinearity to the output of a network layer using a mathematical operation. We will use two types of activation function in this project,
 - a. **Rectified Linear Unit or ReLU Linear activation:** (This is a dummy activation function without any nonlinearity implemented for convenience) It is a piecewise linear function defined as

$$ReLU(Z) = \max(0, Z)$$

Hint: Use [`numpy.maximum`](#)

- b. **ReLU - Gradient:** The gradient of $ReLU(Z)$ is 1 if $Z > 0$ else it is 0.
2. **Linear activation and its derivative:** There is no activation involved here. It is an identity function.

$$Linear(Z) = Z$$

3. **Softmax Activation and Cross- entropy Loss Function:** The softmax activation is computed on the outputs from the last layer and the output label with the maximum probability is predicted as class label. The softmax function can also be referred to as a normalized exponential function which takes a vector of n real numbers as input, and normalizes it into a probability distribution consisting of n probabilities proportional to the exponentials of the input numbers.
4. **Derivative of the softmax_cross_entropy_loss(.):** Define a function that computes the derivative of the softmax activation and cross-entropy loss

Section 2

We will initialize the network and define forward and backward propagation through a single layer. We will extend this to multiple layers of a network. We will initialize and train the multi-layer neural network

1. **Parameter Initialization:** Define a function that can initialize the parameters of the multi-layer neural network. The network parameters will be stored as dictionary elements that can easily be passed as function parameters while calculating gradients during back propagation.
 - a. The weight matrix is initialized with random values from a normal distribution with variance 1. For example, to create a matrix w of dimension 3×4 , with values from a normal distribution with variance 1, we write $w = 0.01 * np.random.randn(3,4)$. The 0.01 is to ensure very small values close to zero for faster training.
 - b. Bias values are initialized with 0. For example a bias vector of dimensions 3×1 is initialized as $b = np.zeros((3,1))$

The dimension for weight matrix for layer $(l + 1)$ is given by (Number-of-neurons-in-layer- $(l+1)$ $\times$ Number-of-neurons-in-layer- l). The dimension of the bias for for layer $(l+1)$ is (Number-of-neurons-in-layer- $(l+1)$ $\times$ 1)

2. **Forward Propagation Through a Single Layer:** If the vectorized input to any layer of a neural network is A_{prev} and the parameters of the layer are given by (W, b) the output of the layer (before the activation is):

$$Z = W.A_{prev} + b$$

3. **Activation After Forward Propagation:** The linear transformation in a layer is usually followed by a nonlinear activation function given by,

$$Z = W.A_{prev} + b$$

$$A = \sigma(Z)$$

Depending on the activation chosen for the given layer, the $\sigma(.)$ can represent different operations.

4. **Multi-Layers Forward Propagation:** Multiple layers are stacked to form a multi-layer network. The number of layers in the network can be inferred from the size of the *parameters* variable from `initialize_network()` function(Step-1). If the number of items in the dictionary element *parameters* is $2L$, then the number of layers will be L .
5. **Backward Propagation through single layer:** Consider the linear layer $Z = W.A_{prev} + b$. We would like to estimate the gradients $\frac{dL}{dW}$ - represented as dW , $\frac{dL}{db}$ - represented as db , and $\frac{dL}{dA_{prev}}$ - represented as dA_{prev} . The input to estimate these derivatives is $\frac{dL}{dZ}$ - represented as dZ . The derivatives are given by,

$$dA_{prev} = W^T dZ$$

$$dW = dZ A^T$$

$$db = \sum_{i=1}^m dZ^{(i)}$$

where, $dZ = [dz^{(1)}, dz^{(2)}, \dots, dz^{(m)}]$ is $(n \times m)$ matrix of derivatives.

6. **Multi-layer Back Propagation:** We have defined the required functions on the notebook to handle backpropagation for a single layer and will stack the layers together and perform back propagation on the entire network.
7. **Prediction:** You will perform forward propagation through the entire network and determine the class predictions for the input data
8. **Parameter Update using Batch-Gradient:** Define a function to update the network parameters with gradient descent.

Inputs:

Parameters: dictionary of network parameters {"W1":[..], "b1":[..], "W2":[..], "b2":[..], ...}

gradients: dictionary of gradient of network parameters
{"dW1":[..], "db1":[..], "dW2":[..], "db2":[..], ...}

epoch: epoch number

alpha: step size or learning rate

Outputs:

parameters: updated dictionary of network parameters

{"W1":[..], "b1":[..], "W2":[..], "b2":[..], ...}

9. **Neural network:** Assemble all the components of the neural network together and define a complete training loop for a Multi-layer Neural Network.
 - a. Define a function that creates the multilayer network and trains the network
 - b. Forward Propagation:

- i. Input 'A0' and 'parameters' into the network using `multi_layer_forward()` and calculate the output of last layer 'A' (before softmax) and obtain cached activations as 'caches'
 - ii. Input 'A' and ground truth labels 'Y' to `softmax_cros_entropy_loss(.)` and estimate activations 'AL', 'softmax_cache', and 'loss'
- c. Backward Propagation:
 - i. Estimate gradient 'dAL' with `softmax_cros_entropy_loss_der(.)` using ground truth labels 'Y' and 'softmax_cache'
 - ii. Estimate 'gradients' with `multi_layer_backward(.)` using 'dAL' and 'parameters'
 - iii. Estimate updated 'parameters' and updated learning rate 'alpha' with `update_parameters(.)` using 'parameters', 'gradients', loop variable 'ii' (epoch number) and 'learning_rate'. Note: Use the same variable 'parameters' as input and output to the `update_parameters(.)` function.

10. **Training** : We will now initialize a neural network with 1 hidden layer whose dimensions is 200. Since the input samples are of dimension 28×28 , the input layer will be of dimension 784. The output dimension is 10 since we have a 10 category classification. We will train the model and compute its accuracy on both training and test sets and plot the training cost (or loss) against the number of iterations.

Note: Most of the functions for the steps above are provided for you in your notebook to make it a little easier.

Submission Directions for Project Deliverables

Learners are expected to work on the project individually. Ideas and concepts may be discussed with peers or other sources can be referenced for assistance, but the submitted work must be entirely your own.

You must complete and submit your work through zyBooks's zyLabs to receive credit for the project:

1. To get started, use the provided Jupyter Notebook in your workspace.
2. All necessary datasets are already loaded into the workspace.
3. Execute your code by clicking the “**Run**” button in top menu bar.
4. When you are ready to submit your completed work, click on “**Submit for grading**” located on the bottom left from the notebook.
5. You will know you have completed the project when feedback appears below the notebook.
6. If needed: to resubmit the project in zyLabs
 - a. Edit your work in the provided workspace.

- b. Run your code again.
- c. Click “**Submit for grading**” again at the bottom of the screen.

Your submission score will automatically be populated from zyBooks into your course grade. However, the course team will review submissions after the due date has passed to ensure grades are accurate.

Evaluation

This project is auto-graded. There are a total of nine (9) test cases and each has points assigned to it. Please review the notebook to see the points assigned for each test case. A percentage score will be passed to Canvas based on your score.